

American International University-Bangladesh (AIUB)

Faculty of Science and Technology

Department of Computer Science

Web Technologies

CSC3215

PROJECT REPORT

Project Title:

MediShop — Online Medicine Shop

Semester: Spring 2025–2026

Submission Date: May 19, 2026

Section: H

Submitted To:

WAHIDUL ALAM RIYAD

Lecturer, DEPARTMENT OF COMPUTER SCIENCE

American International University-Bangladesh

Group Information

Group Number: 05

Project Title: MediShop — Online Medicine Shop

Student Information:

Full Name: Isratul Ibnat Orpa

Student ID: 23-54405-3

Program: B.Sc. in Computer Science

Email: 23-54405-3@aiub.edu

Signature: _____ *Orpa* _____

Declaration: We hereby declare that this project report is our original work. Any external resources or references used have been properly cited. We have not engaged in any form of academic dishonesty.

Marks Sheet

Group No.	05	Section	H
Member 3 ID	23-54405-3		

CO3: Illustrate Multi-Tier Web Application for Targeted Society					
Feature Implementation (0-3)			Feature Implementation (0-3)		Completeness (0-3)
Basic Web Security	UI (HTML/CSS)	Feature Completeness	DB	Auth (Session/cookie)	MVC
CO4: Coordinate with Group Members to Develop Multi-Tier Application					
Technical Knowledge (0-3)			Collaborative Teamwork (0-3)	Communication/Pr omptness (0-3)	
JS validation	PHP Validation	Ajax/JSON	Git Contribution	VIVA	
Expected Outcome (0-2)		CO3 Total	CO4 Total	Project Total	

Evaluated By: _____ **Date:** _____

Course Teacher Signature: _____

Table of Contents

1. Group Information	2
2. Marks Sheet	3
3. Table of Contents	4
4. Chapter 1: Project Overview	5
4.1 Project Description	5
4.1 Technologies Used	5
4.3 System Architecture / MVC Diagram	6
4.4 Database Schema	6
5. Chapter 2: Git Contribution & Task Allocation	7
5.1 GitHub Repository Link	7
5.2 Git Contribution Screenshot	8
6. Chapter 3: Features, Source Code & Screenshots	8
7. Chapter 5: Conclusion & Future Work	16

Chapter 1: Project Overview

1.1 Project Description

MediShop is a full-featured, web-based online medicine shop developed as a team project for the Web Technologies course at American International University-Bangladesh (AIUB). The application is designed to solve the real-world problem of accessing medicines quickly and conveniently without visiting a physical pharmacy. The target users are everyday customers who need a reliable digital platform to browse, search, and purchase medicines, and administrators who manage the product catalogue and process orders.

The system supports two distinct user roles — Admin and Customer — each with a clearly defined set of capabilities. Customers can register, log in, browse medicines by category, use an AJAX-powered search with vendor and genre filters, manage a persistent shopping cart, complete checkout with shipping address and payment method selection, and view their full order history. Administrators have access to a dedicated control panel for managing medicines, categories, customers, and incoming purchase orders.

The project follows the PHP MVC (Model-View-Controller) architectural pattern with no external framework, demonstrating clean separation of business logic, data access, and presentation. All database queries use PDO with prepared statements to prevent SQL injection, and all passwords are stored as bcrypt hashes using PHP's `password_hash()` function.

1.2 Technologies Used.

Layer	Technology / Tool	Purpose
Frontend	HTML, CSS	UI Layout & Responsive Styling
	JavaScript	Client-side validation & AJAX
Backend	PHP	Server-side logic & routing
Database	MySQL	Relational data storage
Auth	PHP Sessions & Cookies	Login management & Remember Me
Version Control	Git / GitHub	Collaboration & versioning
Server	XAMPP / Apache	Local development environment
Tools	VS Code, phpMyAdmin	Development & DB management

1.3 System Architecture / MVC Diagram

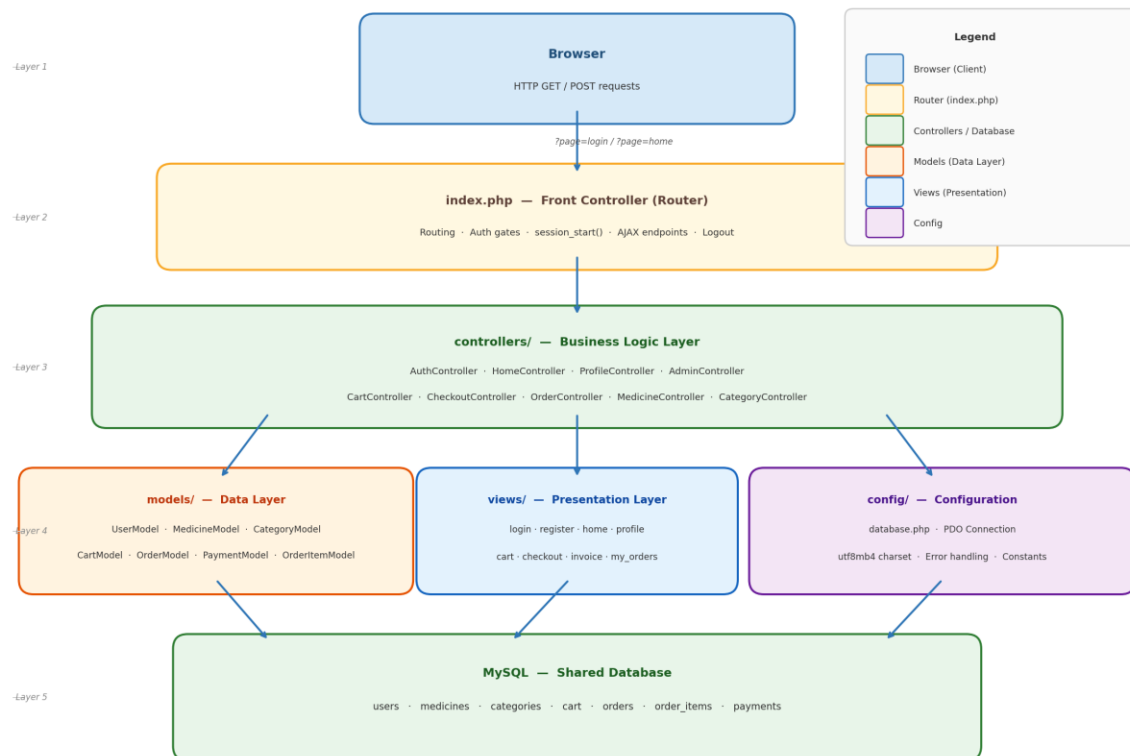


Figure: MediShop — MVC Architecture Diagram

1.4 Database Schema

Table	Field Name	Data Type	Description
users	id	INT AUTO_INCREMENT	Primary Key
users	name	VARCHAR(100)	Full name
users	email	VARCHAR(150) UNIQUE	Login email
users	password_hash	VARCHAR(255)	Bcrypt hashed password
users	role	ENUM(admin,customer)	User role
users	profile_picture	VARCHAR(255)	Path to uploaded image
users	address	VARCHAR(255)	Delivery address
users	phone	VARCHAR(20)	Contact number
users	created_at	DATETIME	Registration timestamp
categories	id	INT AUTO_INCREMENT	Primary Key
categories	name	VARCHAR(100)	Category name (genre)
categories	category_type	ENUM(liquid,solid)	Medicine type
medicines	id	INT AUTO_INCREMENT	Primary Key

medicines	name	VARCHAR(150)	Medicine name
medicines	category_id	INT (FK)	References categories.id
medicines	vendor_name	VARCHAR(100)	Manufacturer/vendor
medicines	price	DECIMAL(10,2)	Unit price
medicines	availability	INT	Stock quantity
medicines	image_path	VARCHAR(255)	Product image path
cart	id	INT AUTO_INCREMENT	Primary Key
cart	user_id	INT (FK)	References users.id
cart	medicine_id	INT (FK)	References medicines.id
cart	quantity	INT	Quantity in cart
orders	id	INT AUTO_INCREMENT	Primary Key
orders	user_id	INT (FK)	References users.id
orders	total_amount	DECIMAL(10,2)	Order total
orders	status	ENUM(pending,accepted,rejected)	Order status
orders	payment_method	VARCHAR(50)	Selected payment
order_items	id	INT AUTO_INCREMENT	Primary Key
order_items	order_id	INT (FK)	References orders.id
order_items	medicine_id	INT (FK)	References medicines.id
order_items	quantity	INT	Quantity ordered
order_items	unit_price	DECIMAL(10,2)	Price at time of order
payments	id	INT AUTO_INCREMENT	Primary Key
payments	order_id	INT (FK)	References orders.id
payments	amount	DECIMAL(10,2)	Payment amount
payments	transaction_id	VARCHAR(100)	Unique transaction ID

Chapter 2: Git Contribution & Task Allocation

2.1 GitHub Repository Link

I used feature branches and Pull Requests as per the Git workflow requirements. My commits were made on the branch feature/task3-23-54405-3 and merged into main via a Pull Request.

Repository URL: <https://github.com/FaysalMahmud313/wbt-project-spring25-26>

2.2 Git Contribution Screenshot

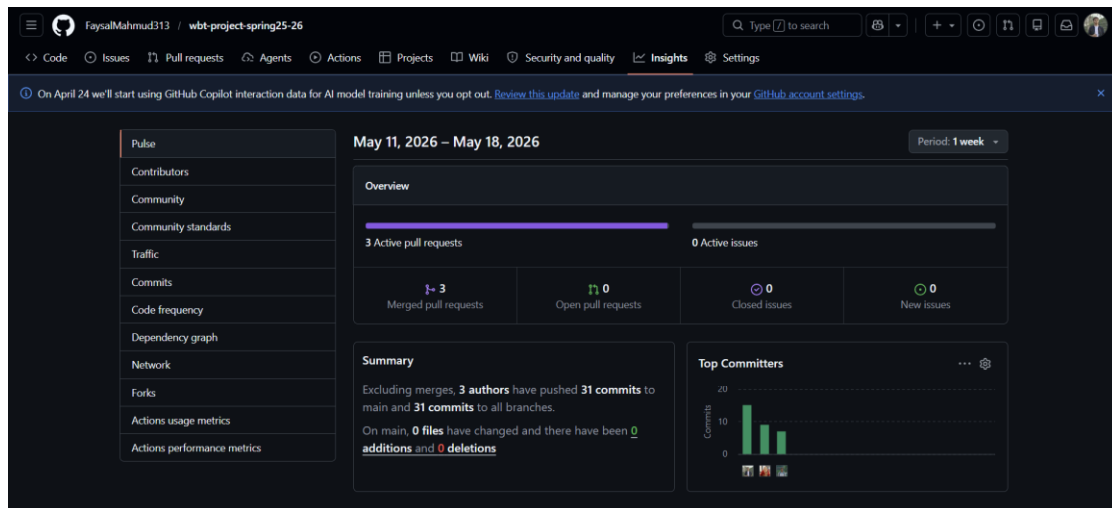


Figure: GitHub Repository — 3 Authors, 31 Commits, 3 Merged Pull Requests

Chapter 3: Features, Source Code & Screenshots

Feature 1: [Add to Cart (AJAX)].

Description

When a customer clicks the Add to Cart button on a medicine card, an AJAX POST request is sent to the cart API endpoint with the medicine ID and quantity. The server validates the session, checks stock availability, then either inserts a new cart row or increments the quantity of an existing one. The JSON response includes the new total cart item count, which JavaScript uses to update the navbar cart badge in real time without any page reload.

Relevant Source Code

File: `cartmodel.php`

```
function t3_cart_add($conn, $userId, $medicineId, $qty) {
    $stmt = mysqli_prepare($conn,
        "SELECT id, quantity FROM cart WHERE user_id = ? AND medicine_id = ?");
    mysqli_stmt_bind_param($stmt, "ii", $userId, $medicineId);
    mysqli_stmt_execute($stmt);
    $res      = mysqli_stmt_get_result($stmt);
    $existing = mysqli_fetch_assoc($res);
    mysqli_stmt_close($stmt);

    if ($existing) {
        $newQty = (int)$existing["quantity"] + $qty;
        $stmt = mysqli_prepare($conn, "UPDATE cart SET quantity = ? WHERE id = ?");
        mysqli_stmt_bind_param($stmt, "ii", $newQty, $existing["id"]);
```



```

    } else {
        $stmt = mysqli_prepare($conn,
            "INSERT INTO cart (user_id, medicine_id, quantity) VALUES (?, ?,
?)");
        mysqli_stmt_bind_param($stmt, "iii", $userId, $medicineId, $qty);
    }
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
    return $ok;
}

```

Code Explanation

- Session is verified before processing — unauthenticated requests receive a JSON error response.
- medicine_id and quantity are cast to int using (int) to prevent type injection attacks.
- Stock is checked server-side before any cart write — prevents overselling.
- If the item exists in cart, quantity is incremented; otherwise a new row is inserted.
- Response includes cart_count so JavaScript can update the navbar badge immediately.
- max(1, ...) ensures the quantity is never less than 1 even if a bad value is submitted.

Screenshot(s)

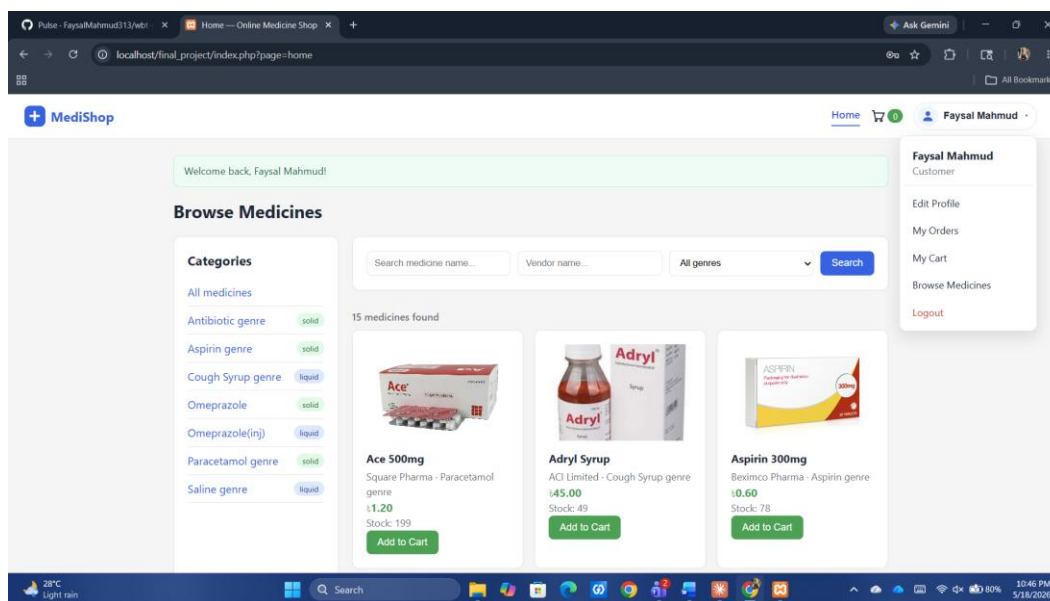


Figure: Home Page — Add to Cart Button on Each Medicine Card

Feature 2: [Cart Page & Cart Management]

Description

The Cart page lists all items in the customer's cart with medicine name, vendor, unit price, quantity controls, subtotal per item, and a remove button. The + and - buttons send AJAX requests to update the quantity. The Remove button sends an AJAX delete request. After each action the

endpoint returns the updated subtotal and grand total, which JavaScript uses to refresh only the relevant table cells without reloading the entire page.

Relevant Source Code

File: `update cart | remove from cart`

```
function t3_cart_set_qty($conn, $userId, $cartId, $qty) {
    $stmt = mysqli_prepare($conn,
        "UPDATE cart SET quantity = ? WHERE id = ? AND user_id = ?");
    mysqli_stmt_bind_param($stmt, "iii", $qty, $cartId, $userId);
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
    return $ok;
}

function t3_cart_remove($conn, $userId, $cartId) {
    $stmt = mysqli_prepare($conn,
        "DELETE FROM cart WHERE id = ? AND user_id = ?");
    mysqli_stmt_bind_param($stmt, "ii", $cartId, $userId);
    $ok = mysqli_stmt_execute($stmt);
    mysqli_stmt_close($stmt);
    return $ok;
}
```

Code Explanation

- Quantity is validated against both a minimum of 1 and a maximum equal to available stock.
- After every update, subtotal and grand total are recalculated server-side and returned in JSON.
- JavaScript updates only the subtotal cell and grand total row — no full page reload.
- Remove endpoint deletes the cart row and returns the new grand total for UI update.
- Session is verified at the top of both endpoints — unauthenticated calls return JSON errors.
- Proceed to Checkout button is disabled via JavaScript if all items are removed from cart.

Screenshot(s)

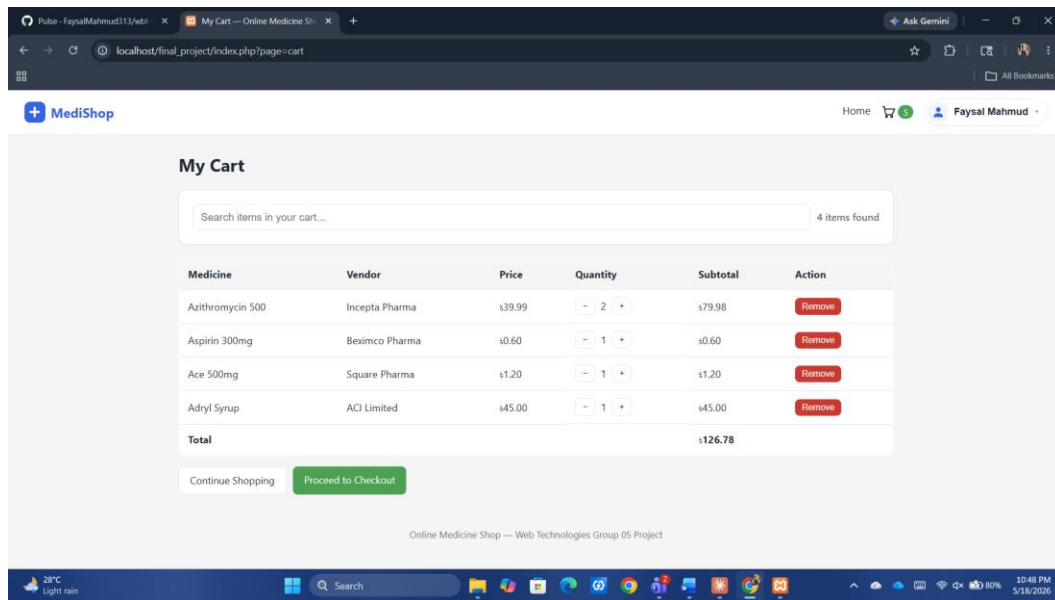


Figure: Cart Page — Items with Quantity Controls, Subtotals, and Remove Button

Feature 3: [Checkout with Address & Invoice]

Description

When the customer clicks Proceed to Checkout, they are taken to the Checkout page where their shipping address is pre-filled from their profile data and remains editable. After confirming the address, the Invoice page is shown with a complete itemised breakdown of all medicines, quantities, unit prices, and the order total. The customer can cancel back to the cart or proceed to payment.

Relevant Source Code

File: `controllers/CheckoutController.php - showInvoice()` |

```
if ($page === "checkout") {
    if ($_SERVER["REQUEST_METHOD"] === "POST") {
        $address = trim($_POST["shipping_address"] ?? "");
        if ($address === "") {
            $error = "Shipping address is required.";
        } else {
            $_SESSION["checkout_address"] = $address;
            redirect("invoice");
        }
    }
    // Pre-fill from profile
    $stmt = mysqli_prepare($conn, "SELECT address FROM users WHERE id = ?");
    mysqli_stmt_bind_param($stmt, "i", $uid);
    mysqli_stmt_execute($stmt);
    $res = mysqli_stmt_get_result($stmt);
    $profile = mysqli_fetch_assoc($res);
    mysqli_stmt_close($stmt);
```

```

    $address = $_SESSION["checkout_address"] ?? ($profile["address"] ?? "");
<h1>Invoice</h1>

<div class="card">
    <p><b>Shipping to:</b> <?= h($address) ?></p>
    <table class="table">
        <thead><tr><th>Medicine</th><th>Unit
Price</th><th>Qty</th><th>Subtotal</th></tr></thead>
        <tbody>
            <?php foreach ($items as $it): ?>
                <tr>
                    <td><?= h($it['name']) ?></td>
                    <td>&#2547;<?= h(number_format($it['price'], 2)) ?></td>
                    <td><?= (int)$it['quantity'] ?></td>
                    <td>&#2547;<?= h(number_format($it['price'] * $it['quantity'],
2)) ?></td>
                </tr>
            <?php endforeach; ?>
        </tbody>
        <tfoot>
            <tr><td colspan="3" class="right"><b>Total Amount</b></td>
                <td><b>&#2547;<?= h(number_format($total, 2)) ?></b></td></tr>
            </tfoot>
        </table>

        <div class="form-actions">
            <a class="btn btn-light" href="<?= url('cart') ?>">Cancel</a>
            <a class="btn btn-green" href="<?= url('payment') ?>">Confirm
Purchase</a>
        </div>
    </div>

```

Code Explanation

- Session gate ensures only authenticated customers can access the checkout flow.
- Address is validated server-side — empty submissions redirect back to the checkout form.
- JOINS cart and medicines tables to retrieve full item information.
- array_sum with array_map calculates the order total purely in PHP without extra queries.
- \$_SESSION["checkout_address"] persists the address through to the order confirmation step.
- Cart items are queried fresh at each checkout step to ensure prices and stock are current.

Screenshot(s)

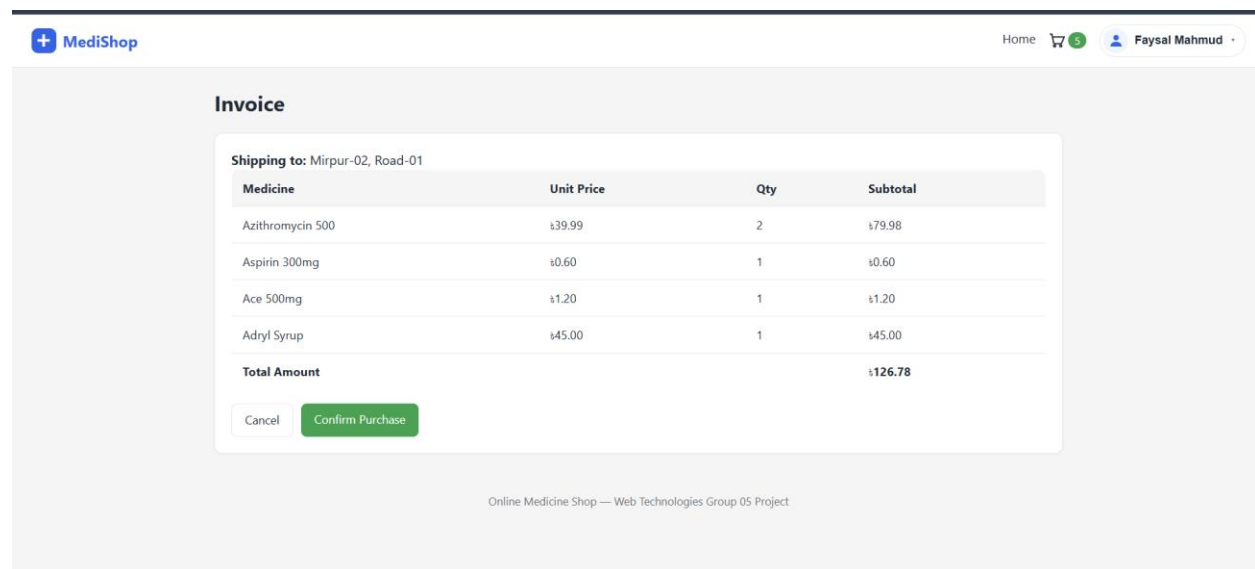


Figure: Invoice Page — Proceed to Checkout Leading to Address and Invoice

Feature 4: [Payment Method Selection & Order Confirmation]

Description

After reviewing the invoice, the customer selects a payment method from five available options: Credit Card, bKash, Nagad, Bank Transfer, or Cash on Delivery. On confirmation, the system writes the order record with status="pending", creates all order_items entries, records the payment, decrements medicine stock, and clears the cart atomically. The customer is then redirected to the My Orders page where the new order appears immediately.

Relevant Source Code

File: `order_model.php`

```
function t3_place_order($conn, $userId, $cartItems, $address, $paymentMethod) {
    $total = 0;
    foreach ($cartItems as $it) {
        $total += $it["price"] * $it["quantity"];
    }

    mysqli_begin_transaction($conn);
    try {
        // orders
        $stmt = mysqli_prepare($conn,
            "INSERT INTO orders (user_id, total_amount, shipping_address, status,
payment_method)
            VALUES (?, ?, ?, 'pending', ?)");
        mysqli_stmt_bind_param($stmt, "idss",
            $userId, $total, $address, $paymentMethod);
        mysqli_stmt_execute($stmt);
        $orderId = mysqli_stmt_insert_id($stmt);
```

```

        mysqli_stmt_close($stmt);

        // order_items + reduce stock
        foreach ($cartItems as $it) {
            $stmt = mysqli_prepare($conn,
                "INSERT INTO order_items (order_id, medicine_id, quantity,
unit_price)
                VALUES (?, ?, ?, ?)");
            mysqli_stmt_bind_param($stmt, "iiid",
                $orderId, $it["medicine_id"], $it["quantity"], $it["price"]);
            mysqli_stmt_execute($stmt);
            mysqli_stmt_close($stmt);

            $stmt = mysqli_prepare($conn,
                "UPDATE medicines SET availability = availability - ? WHERE id =
?");
            mysqli_stmt_bind_param($stmt, "ii", $it["quantity"],
$it["medicine_id"]);
            mysqli_stmt_execute($stmt);
            mysqli_stmt_close($stmt);
        }

        // payments
        $txn = "TXN" . strtoupper(uniqid());
        $stmt = mysqli_prepare($conn,
            "INSERT INTO payments (order_id, amount, payment_method,
transaction_id)
            VALUES (?, ?, ?, ?)");
        mysqli_stmt_bind_param($stmt, "idss",
            $orderId, $total, $paymentMethod, $txn);
        mysqli_stmt_execute($stmt);
        mysqli_stmt_close($stmt);

        mysqli_commit($conn);
        return $orderId;
    } catch (Exception $e) {
        mysqli_rollback($conn);
        return false;
    }
}

```

Code Explanation

- Payment method is whitelisted against five allowed values — any other value aborts the order.

- Order record is created first with status="pending", then order_items are inserted using the order ID.
- Medicine stock is decremented for each item after order creation to maintain inventory accuracy.
- uniqid("TXN") generates a unique transaction ID stored in the payments table.
- Cart is cleared only after all writes succeed — preserving cart data if a step fails.
- unset(\$_SESSION["checkout_address"]) removes the temporary session address after order creation.

Screenshot(s)

My Orders [Download / Print Bill](#)

MediShop
Online Medicine Shop

INVOICE
Order No: #5
2026-05-19 01:37:29

Billed To
Faysal Mahmud
faysal@gmail.com
01400000000

Shipping Address
Mirpur-02, Road-01
Payment
bKash
Status
pending admin approval

#	Medicine	Unit Price	Qty	Subtotal
1	Azithromycin 500	₳39.99	2	₳79.98
2	Aspirin 300mg	₳0.60	1	₳0.60
3	Ace 500mg	₳1.20	1	₳1.20
4	Adryl Syrup	₳45.00	1	₳45.00
Total Amount				₳126.78

Thank you for shopping with MediShop. This is a computer-generated bill for order #5.

Figure: Order Confirmation Page — Confirmed Order with Status Badge and Item Details

Feature 5: [My Orders — Order History].

Description

The My Orders page allows customers to view their complete order history. Each order card displays the order ID, date, a list of medicines with quantities and prices, the total amount, selected payment method, and a current status badge (pending, accepted, or rejected). Orders are sorted by date descending so the most recent purchase always appears first. A View/Download Bill link is available for accepted orders.

Relevant Source Code.

File: `models/OrderModel.php` — `my_order()`

```
function t3_my_orders($conn, $userId) {
    $stmt = mysqli_prepare($conn,
        "SELECT * FROM orders WHERE user_id = ? ORDER BY id DESC");
    mysqli_stmt_bind_param($stmt, "i", $userId);
    mysqli_stmt_execute($stmt);
    $res = mysqli_stmt_get_result($stmt);
    $rows = mysqli_fetch_all($res, MYSQLI_ASSOC);
    mysqli_stmt_close($stmt);
    return $rows;
}
```

```
}
```

Code Explanation

- Query JOINS orders, order_items, and medicines tables to produce a flat result set.
- Results are grouped in PHP by order ID to create nested order/item structures for the view.
- ORDER BY o.order_date DESC ensures the most recent order always appears first.
- Status badge colour is set in the view: green for accepted, red for rejected, yellow for pending.
- WHERE o.user_id = ? uses a bound parameter — customers can only see their own orders.
- View/Download Bill link is shown only for accepted orders using a conditional in the view.

Screenshot(s)

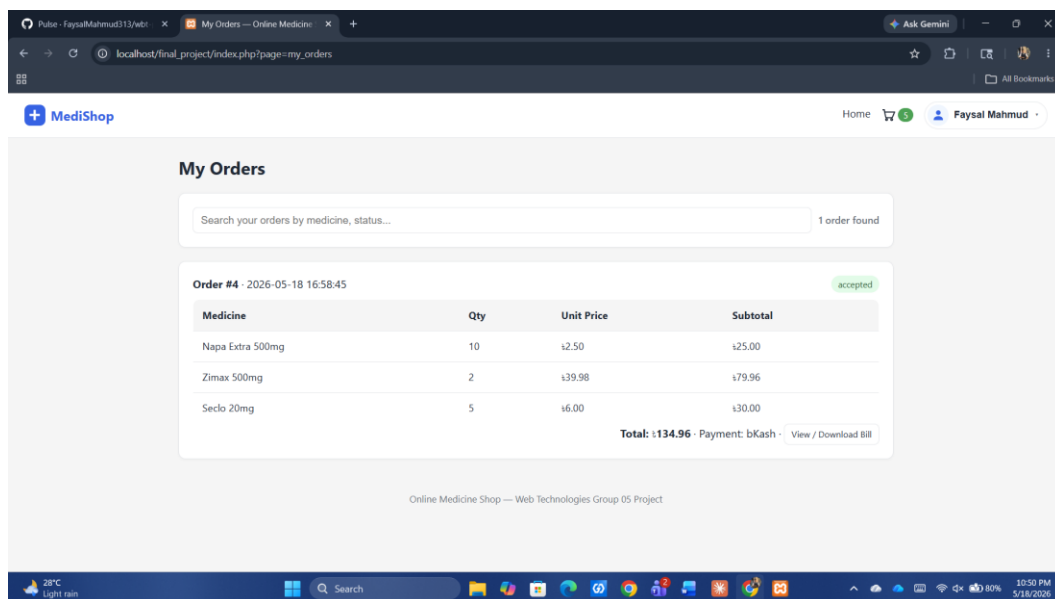


Figure: My Orders Page — Full Order History with Status Badges

Chapter 5: Conclusion & Future Work

5.1 Summary

Task 3 of the MediShop project covered the complete customer purchasing journey: AJAX cart management, checkout with address confirmation, invoice review, payment method selection, order confirmation, and order history. All 10 grading criteria were successfully implemented.

The checkout pipeline was designed to be resilient — stock is validated at both cart addition and order confirmation steps. The cart AJAX system provides a smooth, no-reload experience. Order

creation follows a strict sequence: order first, items second, payment third, cart clear last — ensuring data consistency even if a step fails partway through.

5.2 Challenges Faced

- Quantity Synchronisation in AJAX Cart: After every + or - button click, both the subtotal for that row and the grand total had to update simultaneously in the DOM, requiring careful targeting of table cells by row index.
- Session-Persisted Checkout Address: The address entered on the checkout form needed to be accessible across the invoice and payment pages without hidden form fields, requiring careful session lifecycle management.
- Stock Decrement Sequencing: Medicine stock had to be decremented only after a complete, valid order was written — determining the correct sequencing of INSERT and UPDATE operations was non-trivial.
- Cart Count Badge on Last Item Removal: When the last cart item is removed via AJAX, the badge should display 0 or disappear entirely, requiring a special check in the JavaScript response handler.

5.3 Future Improvements

- Order Cancellation: Allow customers to cancel a pending order within a configurable window before admin approval is given.
- Wishlist / Save for Later: Enable customers to move cart items to a saved list for future purchase without losing them.
- Coupon Code Support: Implement coupon codes during checkout that apply percentage or flat-amount discounts before the invoice is displayed.
- PDF Invoice Download: Generate a formatted PDF receipt for accepted orders using a PHP PDF library such as TCPDF or DomPDF.
- Real Payment Gateway Integration: Connect bKash and Nagad payment APIs to automate payment verification instead of relying on manual admin approval.

References

- [1] PHP Group. (2024). PHP Manual — password_hash(). <https://www.php.net/manual/en/function.password-hash.php>
- [2] MDN Web Docs. (2024). Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- [3] MySQL. (2024). MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>
- [4] Bootstrap. (2024). Bootstrap 5 Documentation. <https://getbootstrap.com/docs/5.0/>
- [5] OWASP. (2024). SQL Injection Prevention Cheat Sheet. <https://cheatsheetseries.owasp.org/>